



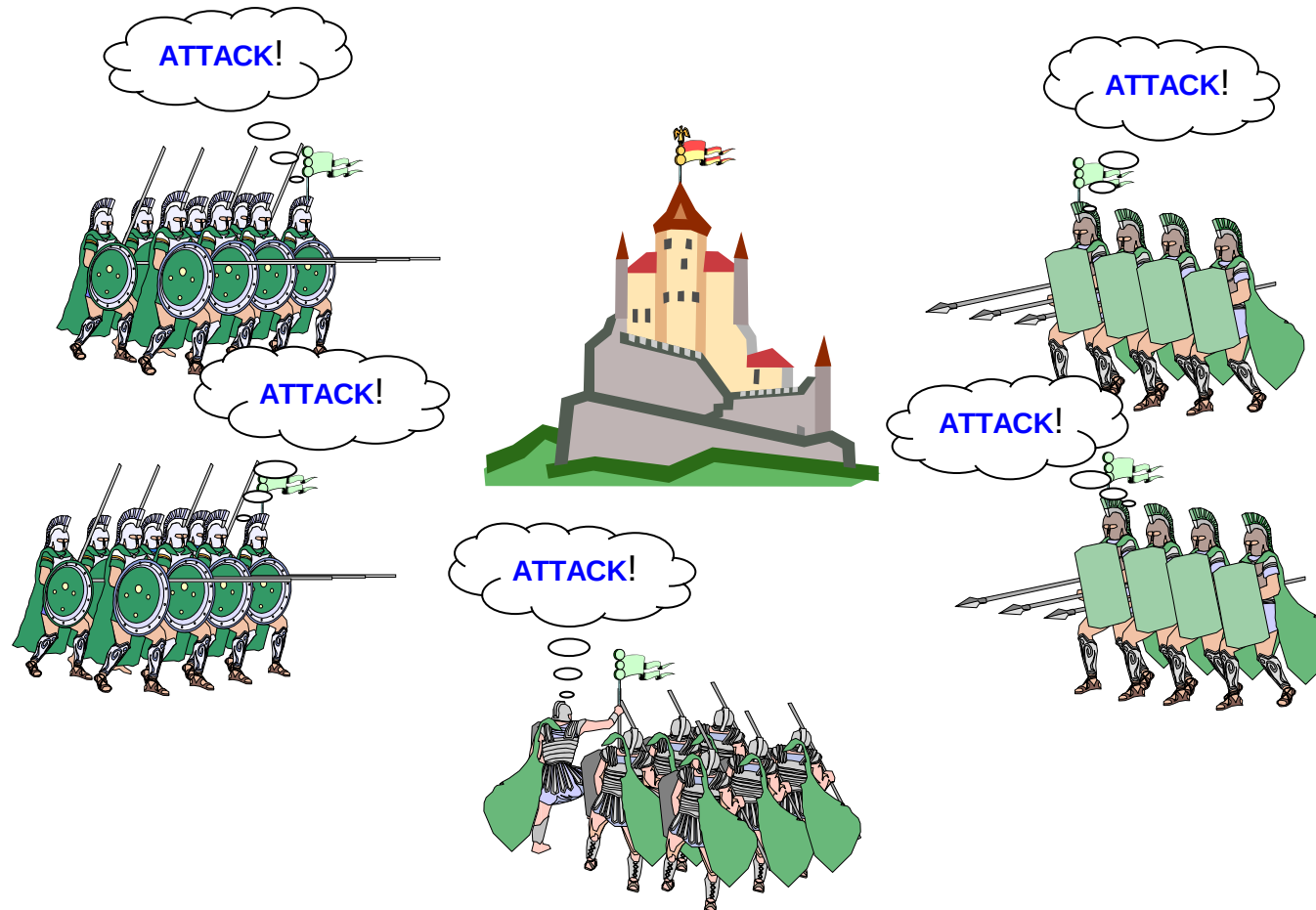
AGREEMENT PROBLEMS

Agreement problems arise in many practical applications:

- agreement on whether to commit or abort the results of a distributed atomic action (e.g. database transaction)
- agreement on an estimate of an airplane's altitude based on the readings of multiple altimeters
- agreement on whether to classify a system component as faulty, given the results of separate diagnostic tests performed by separate processes

COORDINATED ATTACK

A fable:





Basic agreement problem

Battlefield scenario (informally):

- several generals are planning a coordinated attack from different directions against a common objective
- the only way the attack can succeed is if all attack together
- each general has an initial opinion about whether his army is ready to attack
- the generals communicate only via messengers



Ad-hoc protocol:

- each general sends its opinion to all other generals
- they reach agreement based on the total of collected opinions:
 - if most of generals independently decide **ATTACK**, then attack
 - if most of generals independently decide **RETREAT**, then retreat
- let's try a majority based voting:
 - if they are divided, then ... hmm ... well, uh ... hmm ... who cares?
- it may possibly require several rounds
(number of rounds = diameter of the communication graph)

failure-free
environment

Link failures:

- the messengers can be lost or captured
- the protocol does not work anymore!

In fact there is no algorithm that solves this problem correctly.



Formal definition:

1 = **attack** / commit

0 = **retreat** / abort

Termination: All processes eventually decide.

Agreement: No two processes decide on different values

Validity:

1. If all processes start with 0, then 0 is the only possible decision value.
2. If all processes start with 1 and all messages are delivered, then 1 is the only possible decision value.

liveness

safety

Weak Validity version

Validity is quite weak: if even one process starts with value 1, the algorithm is allowed to decide on 1; if all processes start with 1 and all messages are lost the algorithm can make decision on 0.

Even this Weak Validity version of the Coordinated Attack problem is impossible to solve in any distributed system with two or more nodes connected by unreliable channels.

*Even in the **synchronous system** model!*



SYNCHRONOUS SYSTEM $\mathbb{S}^{\text{SYNC}}$ MODEL

- a channel (link) c_{ij} at any time can hold at most one single message $\in \mathbb{M}$
- a *state-transition function* maps (deterministically) **states**_{*i*} and vectors of incoming messages $\in \mathbb{M} \cup \{\text{null}\}$ to **states**_{*i*}
- a *message-generation function* maps **states**_{*i*} x neighbors to elements of $\mathbb{M} \cup \{\text{null}\}$
- execution of the entire system begins with all the processes in arbitrary start states, and all the channels empty
- the processes, in a lock-step manner, repeatedly perform the following 2 steps:
 - step 1) accordingly to the current state generate the messages to be sent to their neighbors; put these messages in the appropriate channels
 - step 2) apply the *state-transition function* to the current state and the incoming messages to make transition to a new state; remove all messages from the channels

(the combination of the 2 steps is called a **round**)



Failures:

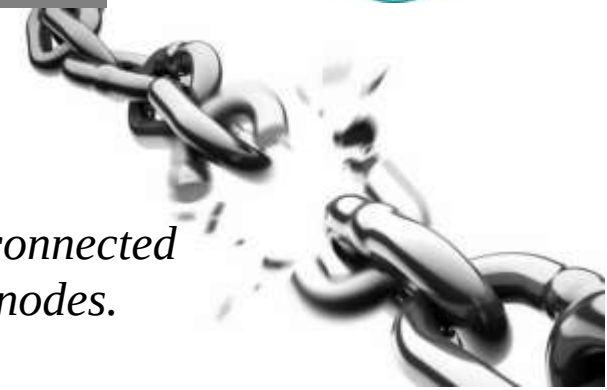
- generally, we can consider both process failures and link failures
- a process can exhibit stopping failure simply by stopping somewhere in the middle of its execution: it might fail before or after step 1 or step 2, or in the middle of performing step 1 (putting only a subset of output messages)
- a process can exhibit Byzantine failure: it can generate its next messages and next state in some arbitrary way independent of *message-generation function* and *state-transition function*
- a link can fail by losing messages (i.e. a process might attempt to place a message in a channel during step 1, but the faulty link might not record the message)

for the following discussion of
the Coordinated Attack we will
consider only link failures



Impossibility Result for link failures

We show the impossibility result for the simplest case of 2 nodes connected by 1 edge. This case implies impossibility for any graph with more nodes.



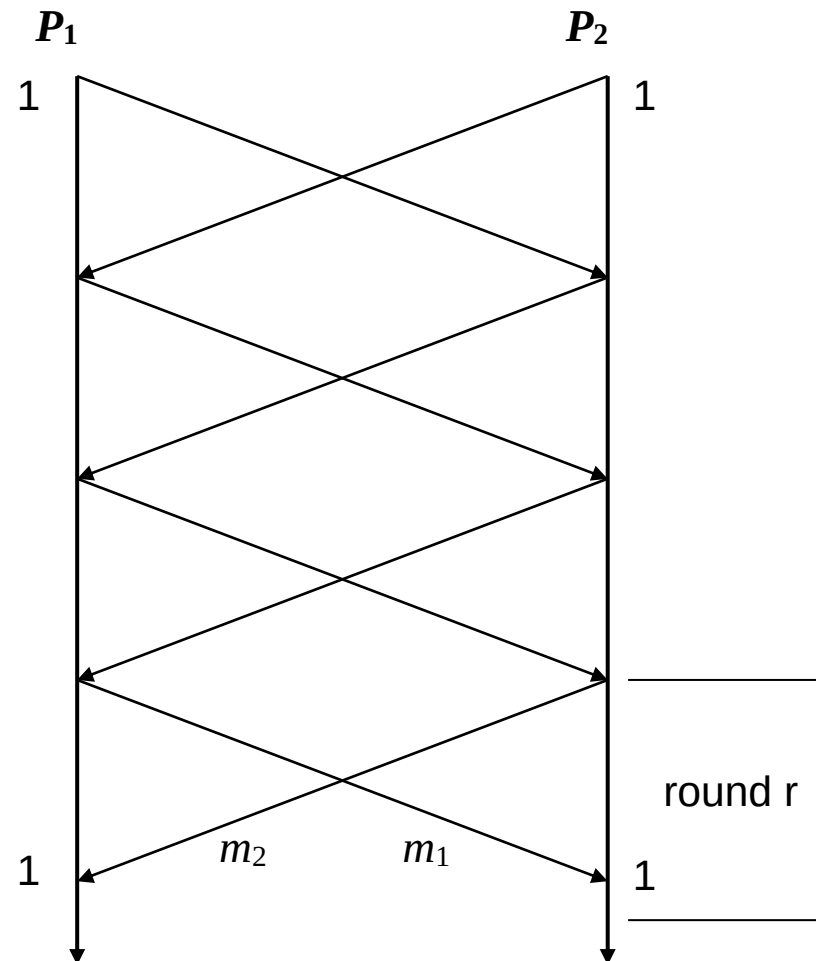
Theorem: Let G be a graph consisting of 2 nodes connected by unreliable edge.
There is no algorithm that solves the Coordinated Attack problem on G .

Proof sketch (by contradiction):

- suppose a solution exists, say algorithm A
- without loss of generality assume: there is only 1 start state for each input value (i.e. only 1 execution possible for a given input and message pattern) and both processes P_1 and P_2 send messages in every round of A
- let α be the execution when both processes start with value 1 and all messages are delivered
- by Termination – both decide (assume at round r), and by Validity – on the value 1

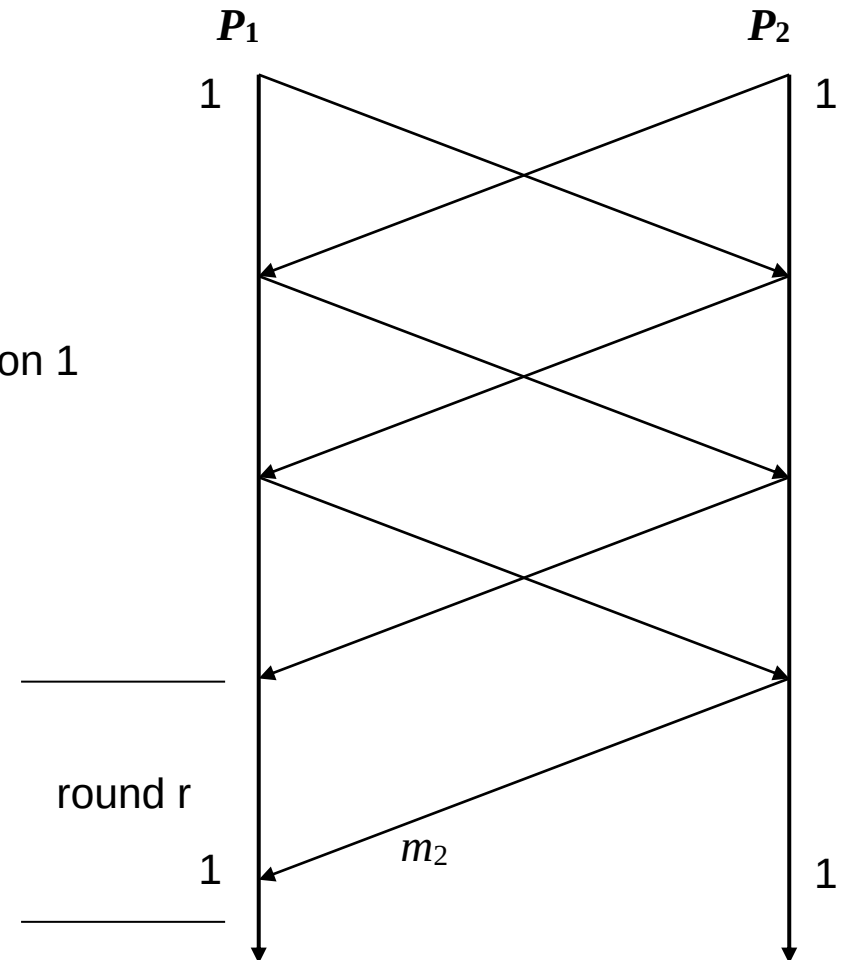


- let α_1 be the same as α , except that after round r all messages are lost
(note, in α_1 both processes also decide on 1)



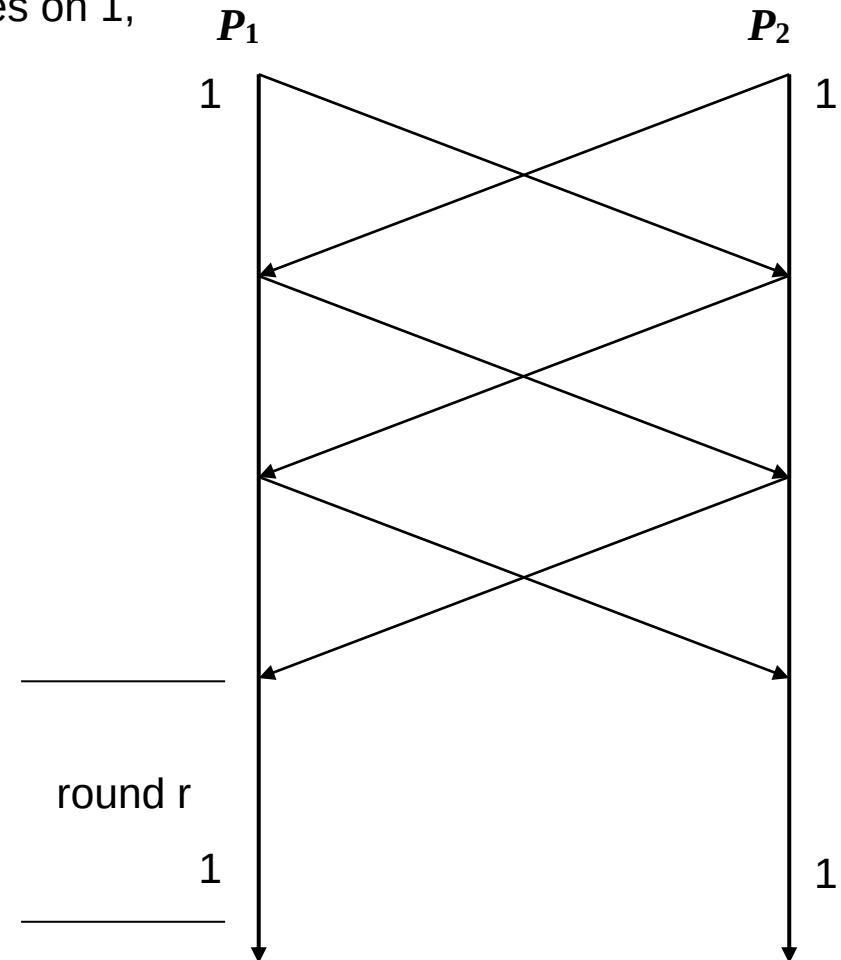


- let α_2 be the same as α_1 , except that m_1 is lost
- α_2 is indistinguishable from α_1 to P_1 ($\alpha_2 \sim^1 \alpha_1$)
- since P_1 decides on 1 in α_1 , it decides on 1 in α_2
- by Termination and Agreement – P_2 also decides on 1





- let's continue: m_2 is lost in $\alpha_3 \sim^2 \alpha_2 - P_2$ still decides on 1, so must do P_1
- ... etc.





- α' = both processes start with 1 and no message is delivered – both decide on 1



- $\alpha'' \sim^1 \alpha' = P_1$ starts with 1, P_2 starts with 0 – P_1 still decides on 1, so must do P_2



- $\alpha''' \sim^2 \alpha'' =$ both start with 0 – P_2 still decides on 1 –
Validity is violated !





OTHER AGREEMENT PROBLEMS

Consensus:

correct

Termination: *all P_c decides exactly 1 value v_i*

Agreement: *all P_c decides the same v_i*

Validity: *v_i was proposed by some P_i*

Every process broadcasts its initial value. Initial values of different processes may be different. All nonfaulty processes agree on *any* common value (we don't care which, as far as the validity condition is not violated). It is irrelevant what value the faulty processes agree on or whether they agree on a value at all. This is a direct generalisation of the Coordinated Attack problem (in fact the Coordinated Attack problem is a binary Consensus).



Byzantine Agreement:

Termination: *all P_c decides exactly 1 value v_s*

Agreement: *all P_c decides the same v_s*

Validity: *if a distinguished source P_s is correct,
then v_s is the value proposed by P_s*

An arbitrary chosen process P_s (source) broadcasts its initial value to other processes. If the source is faulty, then all nonfaulty processes agree on *any* common value.



Interactive Consistency:

Termination: *all P_c agree on 1 vector $\langle v_1, v_2, \dots, v_N \rangle$*

Agreement: *all P_c agree on the same vector $\langle v_1, v_2, \dots, v_N \rangle$*

Validity: *if P_i is correct, i -th vector value v_i was proposed by P_i*

Each process broadcasts its initial value. Initial values of different processes may be different. Processes agree on *all* proposed values. If the i -th process is faulty, then all nonfaulty processes agree on *any* common value for v_i .



For the moment let's
consider failure-free
executions only

Relations among agreement problems

- All agreement problems are closely related.
- For example, the Byzantine Agreement is a special case of the Interactive Consistency problem, in which the initial value of only one process is of interest. $(IC \rightarrow BA)$
- On the other hand, if each of the N processes runs a copy of Byzantine Agreement protocol, the Interactive Consistency problem is solved. $(BA \rightarrow IC)$
- Consensus can be solved using the solution of the Interactive Consistency – all nonfaulty processes can compute the decision basing on the majority value of the common vector, or by choosing the first value of such vector. $(IC \rightarrow C)$
- Thus, solution to the Interactive Consistency and Consensus can be derived from solutions to the Byzantine Agreement. $(BA \rightarrow IC \rightarrow C)$
- It doesn't mean that the Byzantine Agreement problem is weaker than the Interactive Consistency or that the Interactive Consistency is weaker than Consensus. There is no linear ordering.



○ Byzantine Agreement can be solved using Consensus:

1. The source sends its value to all other processes, including itself.
2. All processes run an algorithm for the Consensus problem using the values received in the first step as their proposals.

(C $\rightarrow$ BA)

(If the source is nonfaulty, all the processes will receive the same value in step 1, and all nonfaulty processes will then agree on that value in step 2. If the source is faulty, the other processes may not receive the same value in step 1, but all nonfaulty processes will agree on the same value as a result of the consensus algorithm in step 2.)

Do those
transformations still work
with failure-prone
executions?

APPLICATION FAILURES

COLLEGE APPLICATION

Personal Information

Name (Last)	First	Middle Initial	Home Telephone
Doe	John	J	1123456
Address (Mailing Address)	City	State	Zip
123456 main street	Anytown	TX	75455
E-Mail Address	Other Telephone		
	14187-6543		

Current Income

High School Graduate Or General Education (GED) Test Passed? Yes No

School, Military (Most recent first)	Dates	Credits Earned		Graduate	Degree & Year	Major or Subject
		Quarterly or Semester	Other (Specify)			
Business School				Yes		



CONSENSUS

Impossibility Result for process failures

FLP'85 Impossibility Result

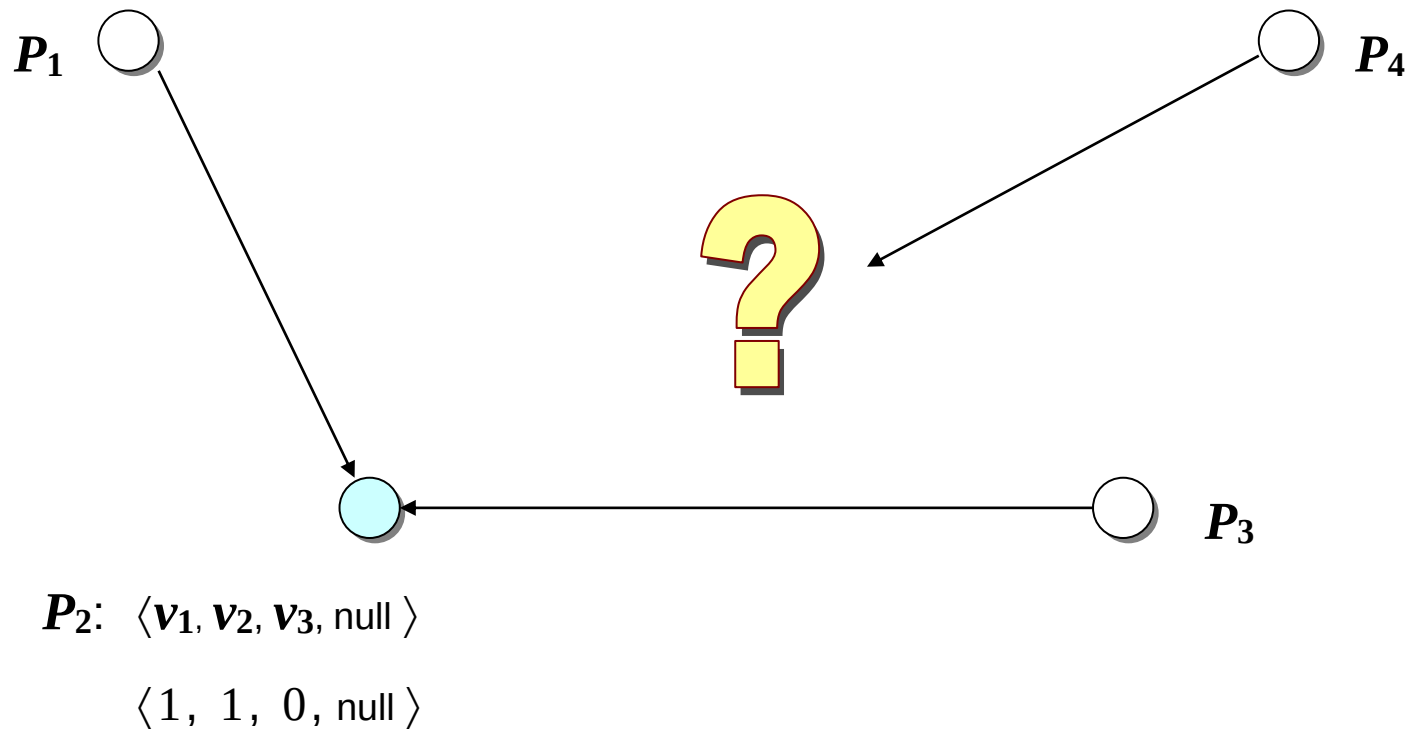
**There is no deterministic Consensus solution
in an asynchronous system
if even a single process only can crash.**

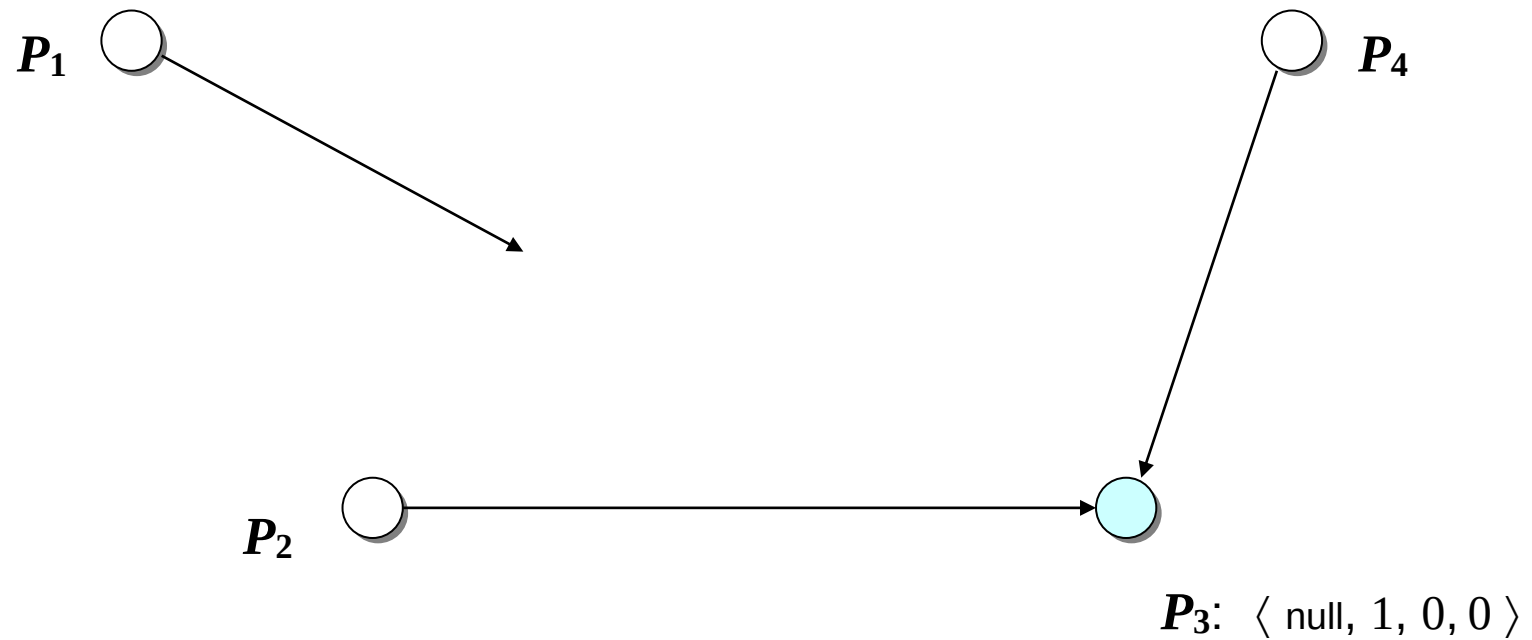
FLP'85 = Fischer-Lynch-Paterson 1985 r. → [1]



Why?

- in asynchronous systems we cannot distinguish a failed process from a slow or very distant one







FLP'85 Impossibility Result

There is no deterministic Consensus solution (1)
in an asynchronous system (2)
if even a single process only can crash. (3)



Despite the impossibility results many non-trivial problems do have solutions, even in asynchronous systems with failures.

Workarounds

For the consensus problem, the FLP'85 result turns out to be very sensitive to the weakening of the model assumptions:

- (1) Randomization – relaxation of the termination condition:

Termination: *every P_c eventually decides with probability 1*

i.e. sacrifice the liveness for safety – e.g. Paxos → [2]

- (2) Stronger synchrony – synchronous (quasi-synchronous) system model
- (3) Weaker failure model – e.g. *initially dead-processes* model



INITIALLY-DEAD PROCESSES

In the model of initially-dead processes, no process can fail after having executed an event.

Def.: A *f-initially-dead fair execution* of N processes is an execution in which at least $N - f$ processes are active, each active process is correct and each message sent to a correct process is delivered.

In a *f-initially-dead fair execution* we can solve consensus as long as $f < \frac{N}{2}$.

General idea:

- because processes do not fail after sending a message, it is safe for P_i to wait for the receipt of a message from P_j if it knows that P_j has already sent at least one message



Algorithm Fischer-Lynch-Paterson [1]

Structures:

- $\text{Fellows}_i, \text{Active}_i, \text{Rcvd}_i$ – sets of process id's; initially $\emptyset$
- $M = \left\lceil \frac{N+1}{2} \right\rceil$ – at least M processes are correct

```
Pi :  bcast (NAME, i)
      while (|Fellowsi| < M-1)
          recv (NAME, j)
          Fellowsi := Fellowsi ∪ {j}
      bcast (PROPOSE, i, vi, Fellowsi)
      Activei := Fellowsi
      while (Activei ⊄ Rcvdi)
          recv (PROPOSE, j, vj, Fellowsj)
          Activei := Activei ∪ Fellowsj ∪ {j}
          Rcvdi := Rcvdi ∪ {j}
      compute_knot()
```

first stage

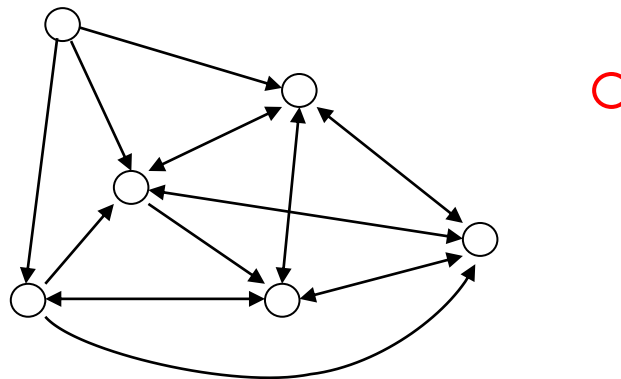
second stage



Description:

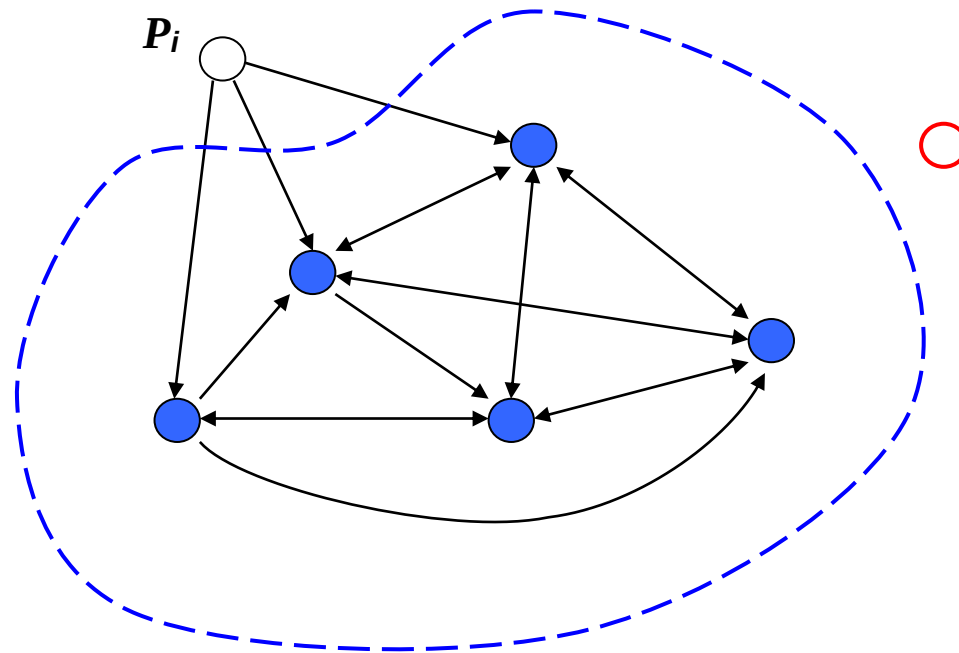
First stage

- processes construct a directed graph $\mathbf{G}$ broadcasting its identity
- the fellows of P_i are such P_j from which P_i has received a NAME message:
edge $\overleftarrow{(i, j)}$ in $\mathbf{G} \Leftrightarrow \text{recv}(P_i, \overleftarrow{P_j}, \text{NAME})$
- P_i waits for the receipt of $M-1$ messages – as there are at least M correct processes, each correct process P_c receives sufficiently many messages to complete this work
- an initially-dead process has not sent any messages – it forms an isolated node in $\mathbf{G}$
- each P_c has $M-1$ fellows – it is not isolated





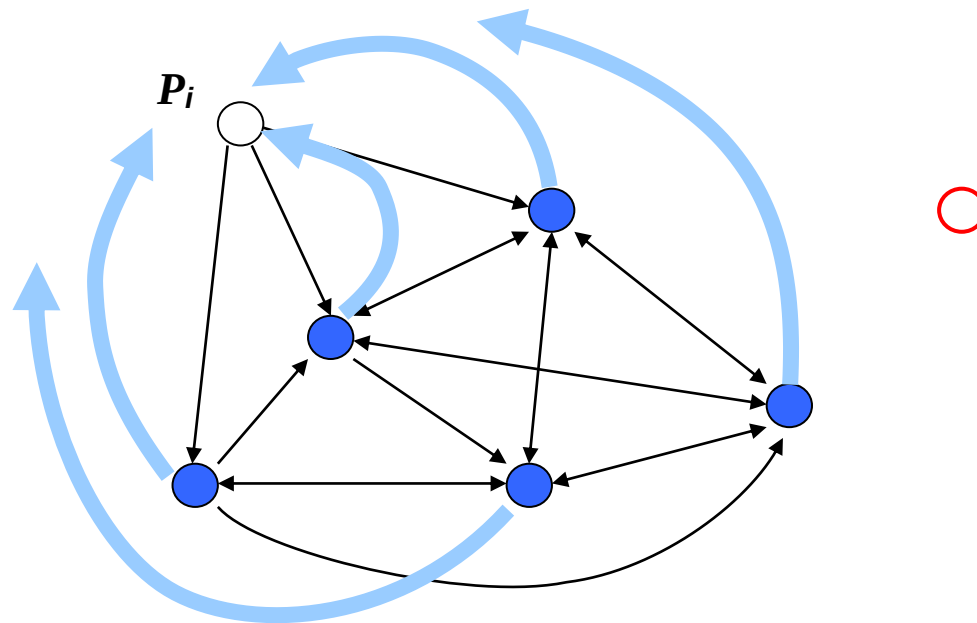
- obviously then, there is a knot in $\mathbf{G}$ containing correct processes, say $\mathbf{K}$,
- and because each P_c has out-degree $M-1$, this knot has size at least M
- consequently there is exactly 1 knot ($2M > N$); not necessarily all P_c belong to $\mathbf{K}$
- as a correct P_i has $M-1$ fellows, at least one fellow belongs to $\mathbf{K}$, implying that all processes in $\mathbf{K}$ are descendants of P_i





Second stage

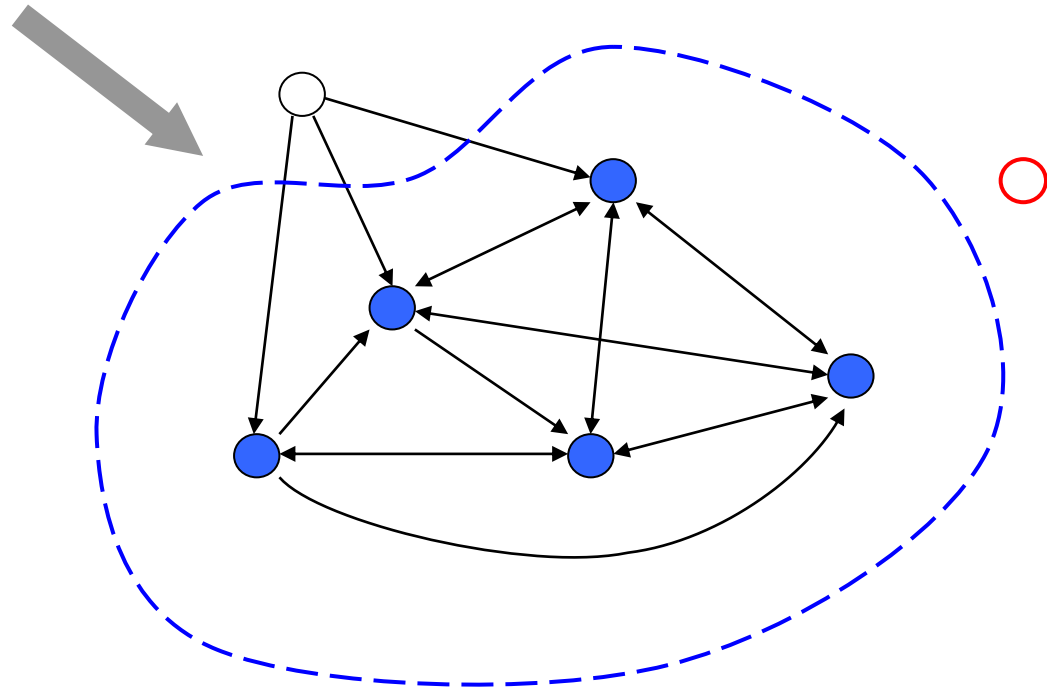
- each P_c constructs its own view of $\mathbf{G}$ by receiving the set of fellows from every process they know to be correct (no deadlock occurs since no P_c can fail now)
- at the end of this stage, each P_c has received the set of fellows of each of its descendants, allowing to compute a unique knot $\mathbf{K}$ in $\mathbf{G}$





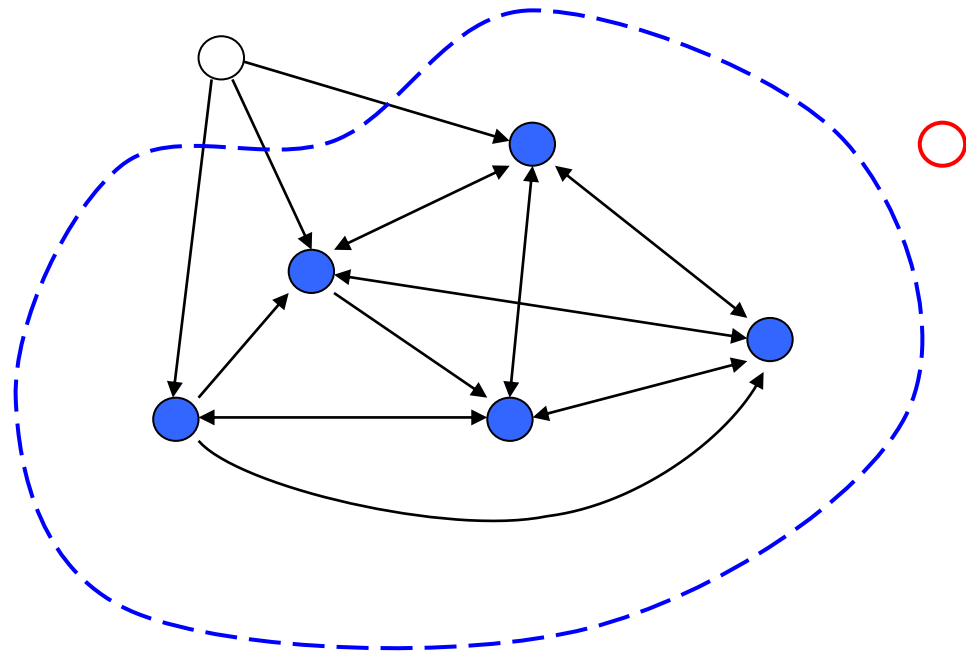
Consensus:

- as all P_c agree on a knot K of correct processes,
- and each of them broadcast its proposed value v_i , together with its fellows,
- after computing K , processes decide on a value in function of collected values from processes in K (majority of values or the lowest value, etc)
- $O(N^2)$ messages required



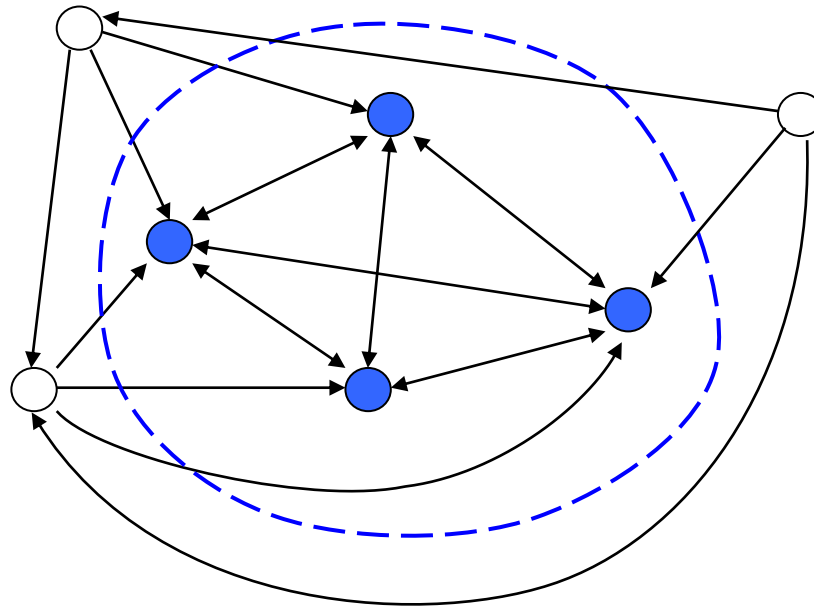


Consensus is computed only among propositions from K !





Another example of graph G:





Yet another popular agreement problems:

k -set consensus

- processes agree on a small set of k values

approximate agreement

- processes agree on values close to one another

renaming

- processes agree on necessarily distinct values



Renaming problem:

The renaming problem assigns to each process P_i , a name x_i from a domain $\mathbb{X}$

Termination: *each correct P_i is eventually assigned a name x_i*

Validity: *the name x_i belongs to $\mathbb{X}$*

Agreement: *for non-faulty processes P_i and P_j : $x_i \neq x_j$*

Anonymity: *the code executed by any process must not depend on its initial identifier*

The renaming problem is useful for name space transformation. A specific example is when processes from different domains need to collaborate, but must first assign themselves distinct names from a small domain.

Another example of renaming is when processes need to use their unique names (IDs) as ordering labels (in a priority queue).



Some other well-know problems related to agreement:

Election

- in the initially dead-processes model, as all processes agree on a knot **K** of correct processes, it is trivial to elect a process
- e.g. a process with the lowest ID in **K** is elected

Election & Consensus

- in the initially dead-processes model, any election algorithm choosing a correct process as leader also solves the Consensus problem:
- the leader broadcasts its initial value and all correct processes decide on it
- however, in the fail-stop model, the availability of a leader does not help – the leader can crash before broadcasting its value
- ...
- anyway, the election is not solvable with crash failures

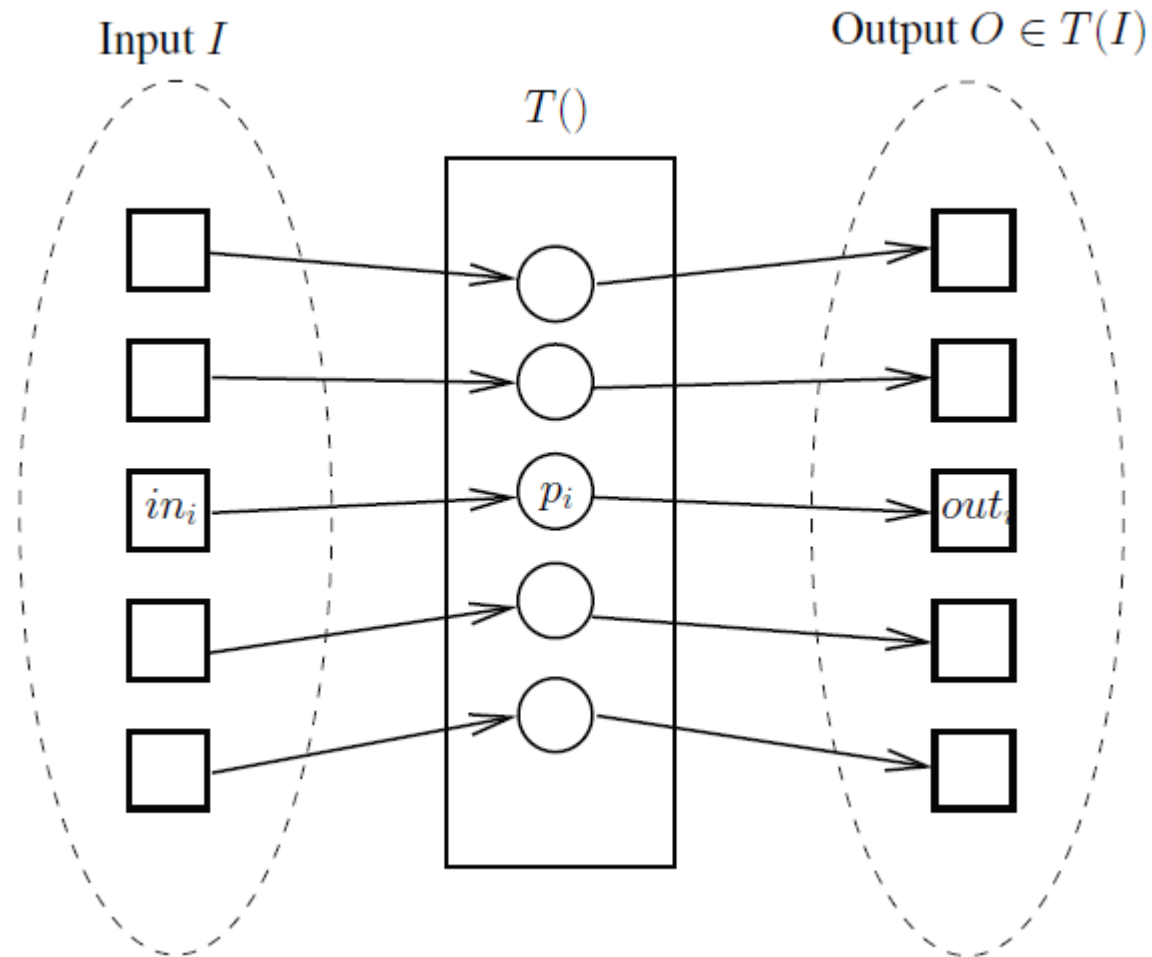


GENERALIZATION:

Def.: A *distributed task* is described by sets In and Out of possible input and output values, and a (possibly partial) function $\mathcal{T}: In^N \rightarrow Out^N$

The interpretation of the mapping $\mathcal{T}$ is that if vector $I = \langle in_1, \dots, in_N \rangle$ describes the input of the processes, then $\mathcal{T}(I)$ is the set of legal outputs of the algorithm – decision vectors described $O = \langle out_1, \dots, out_N \rangle$.

If $\mathcal{T}$ is a partial function, not every combination of input values I is allowed.





Examples

- Consensus – all decisions must be equal:

$$\mathbb{O}_{ut}^N = \{ \langle 0, 0, \dots, 0 \rangle, \langle 1, 1, \dots, 1 \rangle, \dots \}$$

where:

$$\mathcal{T}(\langle 0, 0, \dots, 0 \rangle) = \{ \langle 0, 0, \dots, 0 \rangle \}$$

$$\mathcal{T}(\langle \dots, 5, \dots \rangle) = \{ \dots, \langle 5, 5, \dots, 5 \rangle, \dots \}$$

- Election – one process decide 1 and the others decide 0:

$$\mathbb{O}_{ut}^N = \{ \langle 1, 0, \dots, 0 \rangle, \langle 0, 1, \dots, 0 \rangle, \dots, \langle 0, 0, \dots, 1 \rangle \}$$



Def.: Algorithm A is a f -crash resilient solution for task $\mathcal{T}$ if it satisfies:

Termination: in every f -crash fair execution, all P_c decide

Consistency: if all P_i are correct, the decision vector O is in $\mathcal{T}(I)$



BIBLIOGRAPHY

- [1]  M. J. Fischer, N. A. Lynch, M. S. Paterson, “Impossibility of distributed consensus with one faulty process”. *Journal of the ACM* no. 32, 1985, pp.374-382.
- [2]  L. Lamport, “Paxos made simple”. *ACM SIGACT News* no. 32, vol. 4, 2001, pp.51-58.
- [3]  M. Raynal, " Fault-Tolerant Message-Passing Distributed Systems ", Springer, 2018, ch. 10, 12.
- [4]  G. Tel, “Introduction to Distributed Algorithms”, Cambridge University Press, 2000, ch.14.